

Final_Andrew Claxton

2026-06-24

Fundamentals of Machine Learning Final - Andrew Claxton

Executive Summary: This analysis utilizes machine learning to optimize fuel procurement strategies across power generation facilities. By applying clustering algorithms to EIA-923 data, we evaluated fuel transactions across axis: Fuel Quality (via K-means) and Purchasing Efficiency over time (via DBSCAN). Fuel quality analysis delivered insights on operational risks, potential financial inefficiencies, and actionable pathways for cost reduction and contract renegotiation. DBSCAN clustering was not as effective in delivering insights, as coal purchasing data was not distinct enough to produce clearly discernable groups.

Introduction: The EIA-923 Schedule 2, Part A dataset contains information on each shipment of fuel received at a reporting plan including the source location, quantities, energy content, cost, and fuel quality characteristics. Because the vast majority of the data maturity is categorized as “final” and it doesn’t have any obvious affect on fuel quality or cost, we will be eliminating this variable. The mercury, moisture, and chlorine content variable fields also have a large number of missing values, so we will also eliminate these variables from the analysis. We executed numeric conversion of months and years and a log transformation of the cost per mmbtu to mitigate the effect of a long right tail in the data.

Problem Statement:

1. What are the patterns of coal shipment quality? For this We will use K-means clustering based on the fuel mmbtu per unit, the sulfur content, and the ash content. This works because the variables are continuous and tend not to have major outliers.
2. What are the patterns of coal shipments’ cost per mmbtu, month, year, and coal quality? We will use DBSCAN clustering based on fuel mmbtu per unit, month, year, and cost per mmbtu. This is seemingly the appropriate clustering algorithm because prices can vary greatly, meaning we may have significant outliers and non-spherical clusters.

```
library(dplyr)
library(fastDummies)
library(caret)
library(cluster)
```

```

library(factoextra)
library(dbSCAN)
fuel <- read.csv("C:\\Users\\User\\OneDrive\\Desktop\\Grad School_Business Analytics\\Fundamentals of M
#Read data

set.seed(1993)
sampled_fuel <- fuel %>% sample_frac(0.02)
rm(fuel)
gc()

```

```

##          used (Mb) gc trigger (Mb) max used (Mb)
## Ncells 2573157 137.5   6642675 354.8  4663494 249.1
## Vcells 5485286  41.9   20176906 154.0 25016833 190.9

```

```

#sample 2 percent then free up the space
#by deleting
fuel_coal <- sampled_fuel %>%
  filter(fuel_type_code_pudl == "coal")
#take only rows for coal

quality <- fuel_coal %>%
  select(fuel_mmbtu_per_unit,
         sulfur_content_pct,
         ash_content_pct) %>%
  na.omit()
#Select our "quality columns then omit NAs

quality_scaled <- scale(quality)
#normalize and (below) print a sample to confirm
head(quality_scaled)

```

```

##   fuel_mmbtu_per_unit sulfur_content_pct ash_content_pct
## 1      -0.7396315      -0.8467940      -0.61703027
## 2       1.0335557       2.3159534       0.10340191
## 3       1.3519331      -0.2010014      -0.09404337
## 4       0.4607888       1.7202418      -0.04832945
## 5      -0.6856935      -0.8157992      -0.58165823
## 6       1.3611109      -0.3121263       0.08302884

```

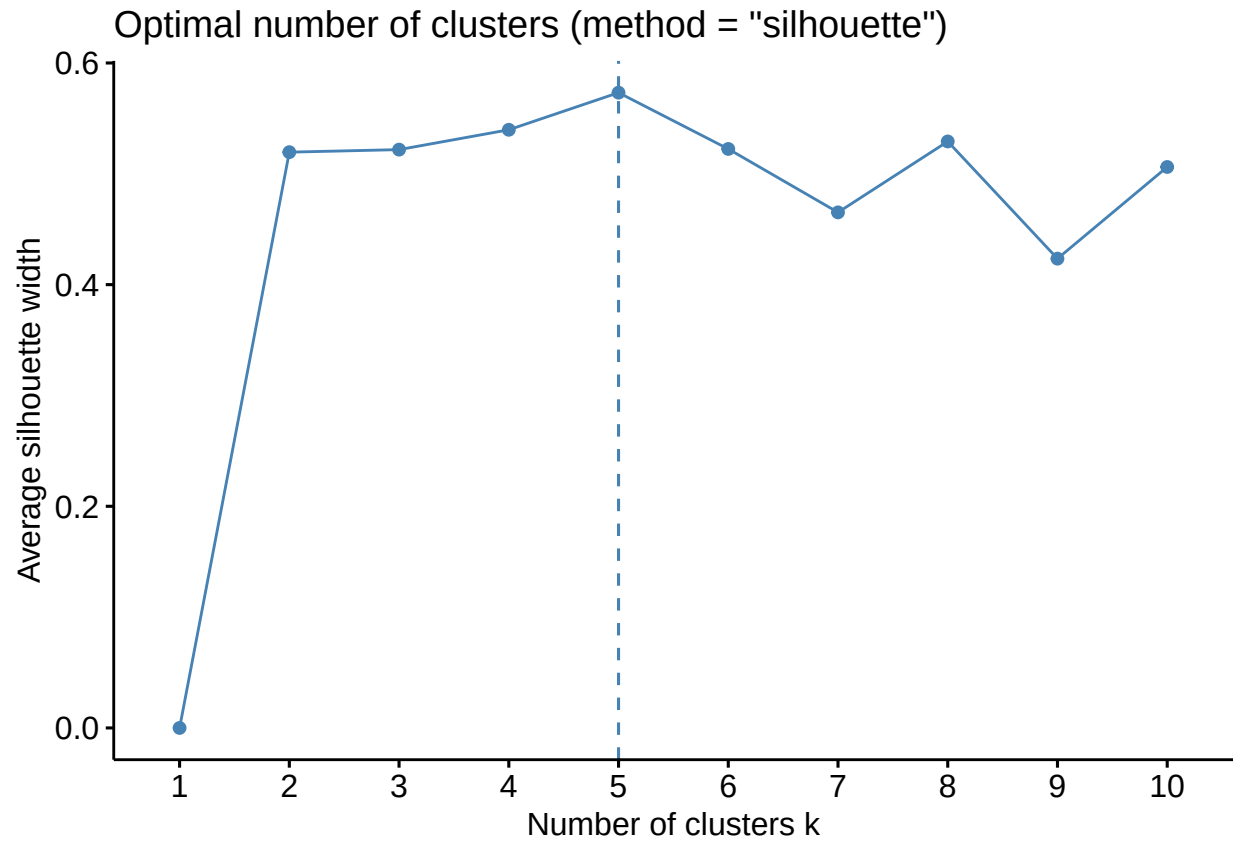
```

train_index <- createDataPartition(
  y = 1:nrow(quality_scaled),
  p = 0.75,
  list = FALSE)
#index with 70/30 data split

train <- quality_scaled[train_index, ]
test  <- quality_scaled[-train_index, ]
#create data frames with train and test

fviz_nbclust(train, kmeans, method = "silhouette")

```

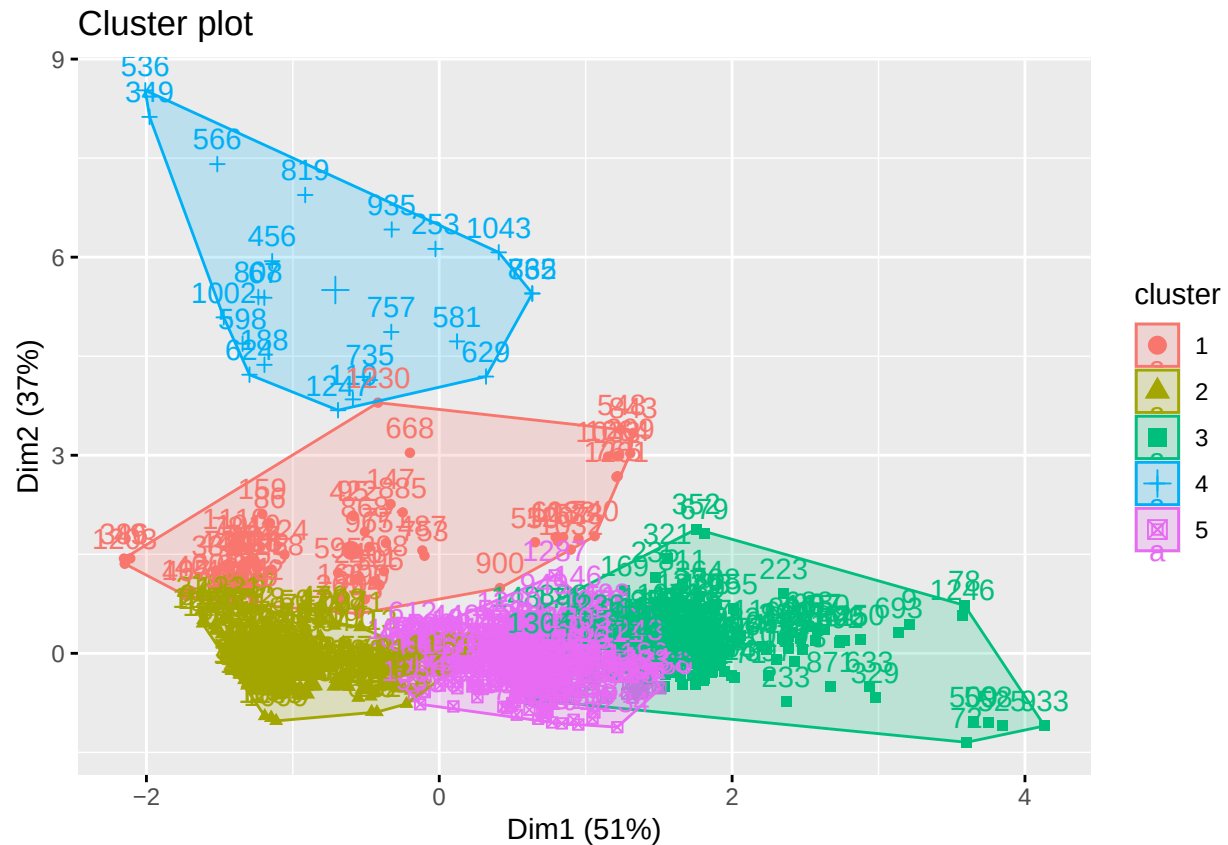


```
#automatically runs K-means for K = 1-10
```

The optimal number of clusters for fuel quality cluster analysis (K-Means) is 5

Fuel Quality Clusters (K-Means)

```
#K = 5  
qualityKM <- kmeans(train, centers = 5, nstart = 25)  
fviz_cluster(qualityKM, data = train)
```



```
#visualize the clusters
```

```
qualityKM$centers
```

```
##   fuel_mmbtu_per_unit sulfur_content_pct ash_content_pct
## 1      -1.1080463      0.03713532      1.33186632
## 2      -0.7618685     -0.73389478     -0.52485262
## 3       0.8213290      1.64823444      0.02159302
## 4     -2.0006959      0.20123824      5.36062863
## 5       1.0178583     -0.08047755      0.10086715
```

```
#look at cluster centroids
```

Test data centroids very closely matches training data centroids

```
test_clusters <- apply(
  test,
  1,
  function(x) {
    which.min(colSums((t(qualityKM$centers) - x)^2))
  }
)
#assign test observations to nearest
```

```
#training centroid
```

```
table(test_clusters)
```

```
## test_clusters
```

```
##   1   2   3   4   5
```

```
## 13 144  65   7  96
```

```
#check cluster sizes
```

```
test_centers <- aggregate(  
  test,  
  by = list(Cluster = test_clusters),  
  mean  
)  
test_centers
```

```
##   Cluster fuel_mmbtu_per_unit sulfur_content_pct ash_content_pct  
## 1       1         -1.3683233         0.07129156         1.38576063  
## 2       2         -0.7396381        -0.73807766        -0.54403997  
## 3       3          0.8689586         1.67785953        -0.03128731  
## 4       4         -1.8005879        -0.46745233         4.91447563  
## 5       5          0.9766250        -0.08906364         0.12533570
```

```
#check centroid
```

Fuel Cost Clustering (DBSCAN)

```
cost <- sampled_fuel %>%  
  filter(fuel_type_code_pudl == "coal") %>%  
  select(  
    report_date,  
    fuel_cost_per_mmbtu,  
    fuel_mmbtu_per_unit  
  )  
# Filter to coal and select variables  
  
cost$fuel_cost_per_mmbtu <- as.numeric(cost$fuel_cost_per_mmbtu)  
#convert cost column to numeric  
  
cost$report_date <- as.POSIXct(cost$report_date)  
cost$Year <- as.numeric(format(cost$report_date, "%Y"))  
cost$Month <- as.numeric(format(cost$report_date, "%m"))  
#create month and year  
cost$report_date <- NULL  
#remove original date field  
cost <- na.omit(cost)
```

```

#omit NAs
cost$fuel_cost_per_mmbtu <-
  log1p(
    cost$fuel_cost_per_mmbtu -
      min(cost$fuel_cost_per_mmbtu, na.rm = TRUE) + 1
  )
#log transform cost per mmbtu
cost_scaled <- scale(cost)
#normalize

```

Elbow Method Plot with $K = 4$ (# of dimensions +1)

```

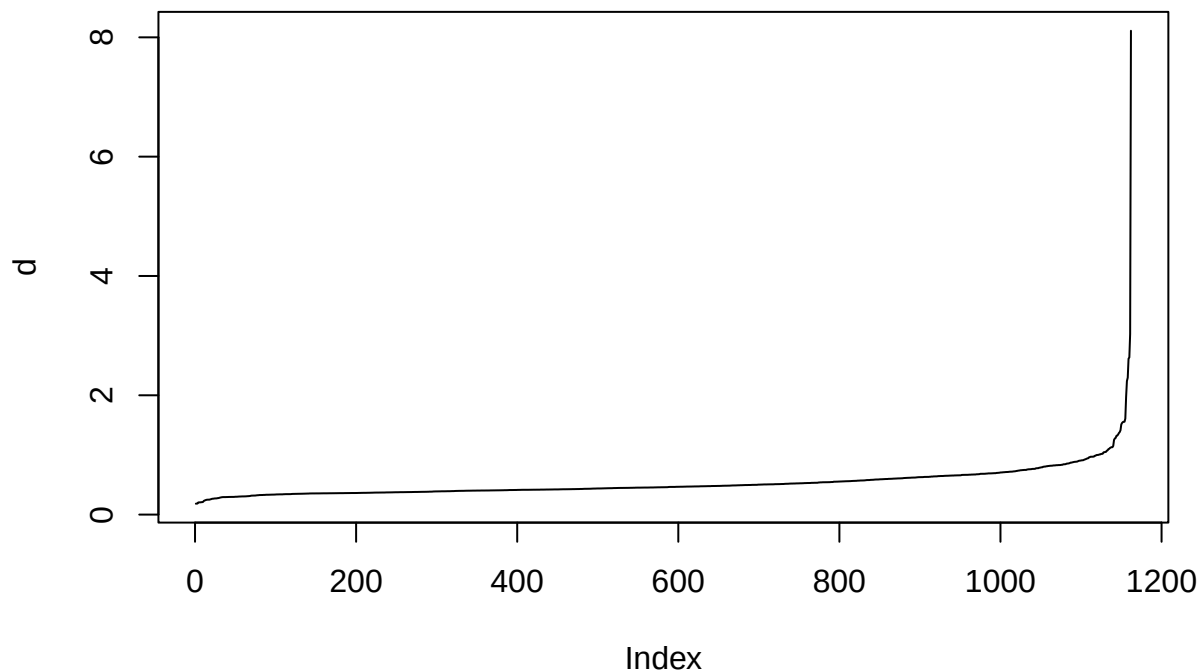
library(FNN)

knn <- get.knn(cost_scaled, k = 4)

d <- sort(knn$nn.dist[, 4])

plot(d, type = "l")

```



```

#look at the elbow to determine epsilon value
#k here is minPoints, NOT number of clusters
#rule of thumb -> min points = # of dimensions +1

```

Cost is affected by extreme outliers on the right tail

```
dbscan(cost_scaled, eps = .5, minPts = 4)
```

```
## DBSCAN clustering for 1162 objects.
## Parameters: eps = 0.5, minPts = 4
## Using euclidean distances and borderpoints = TRUE
## The clustering contains 6 cluster(s) and 266 noise points.
##
##    0    1    2    3    4    5    6
## 266 873    4    5    4    6    4
##
## Available fields: cluster, eps, minPts, metric, borderPoints
```

```
dbscan(cost_scaled, eps = .6, minPts = 4)
```

```
## DBSCAN clustering for 1162 objects.
## Parameters: eps = 0.6, minPts = 4
## Using euclidean distances and borderpoints = TRUE
## The clustering contains 7 cluster(s) and 154 noise points.
##
##    0    1    2    3    4    5    6    7
## 154 985    4    2    4    3    5    5
##
## Available fields: cluster, eps, minPts, metric, borderPoints
```

```
dbscan(cost_scaled, eps = .7, minPts = 4)
```

```
## DBSCAN clustering for 1162 objects.
## Parameters: eps = 0.7, minPts = 4
## Using euclidean distances and borderpoints = TRUE
## The clustering contains 4 cluster(s) and 69 noise points.
##
##    0    1    2    3    4
## 69 1079    4    6    4
##
## Available fields: cluster, eps, minPts, metric, borderPoints
```

There tends to be one major cluster with other very small clusters and 266 outliers with an eps of 0.5

```
db <- dbscan(cost_scaled, eps = 0.5, minPts = 4)

aggregate(
  cost,
  by = list(Cluster = db$cluster),
  mean
)
```

##	Cluster	fuel_cost_per_mmbtu	fuel_mmbtu_per_unit	Year	Month
## 1	0	1.484209	19.43056	2016.429	6.244361
## 2	1	1.457242	20.80850	2013.901	6.738832
## 3	2	1.277051	13.52350	2023.750	10.250000
## 4	3	1.452210	23.33043	2023.200	9.800000
## 5	4	1.464555	22.56839	2024.750	7.000000
## 6	5	1.222746	17.34338	2024.833	2.166667
## 7	6	1.352912	12.55991	2017.000	3.000000

Conclusion

We clustered coal transactions by quality using the K-means clustering algorithm. We chose to fit the data into 5 clusters, which fell into a single moderately-sized high-quality fuel cluster, two small low-quality fuel clusters, and two large moderate quality fuel categories. The clusters were formed by calculated distances between BTU per unit, ash content, and sulfur content. Cluster 4 is of particular interest because the members of this cluster are purchasing particularly low-quality fuel with extremely high ash content. Cluster 5 is also of interest because the members of this cluster are purchasing particularly good fuel quality with low sulfur and ash content. This is useful for determining which power plants are purchasing low-quality fuel that may result in power plant inefficiencies or increased volume of fuel needed to keep up with power demands.

DBSCAN was applied to the coal-only subset of the data using several values of eps to identify naturally occurring density-based clusters. At $\text{eps} = 0.5$, the algorithm identified six clusters, but 266 observations (22.9%) were classified as noise, indicating that many records did not belong to a sufficiently dense region. Increasing eps to 0.6 reduced the number of noise observations to 154 (13.3%) while producing one dominant cluster containing 985 observations and several very small clusters. At $\text{eps} = 0.7$, the majority of the data (1,079 of 1,162 observations, or 92.9%) merged into a single cluster, with only 69 observations classified as noise and two additional clusters containing just four to six observations each. These results suggest that the coal purchase data does not contain multiple well-separated density-based groups. Instead, the observations form one primary dense population with a relatively small number of isolated or unusual records. The results also demonstrate that increasing eps progressively merged smaller clusters into the dominant cluster while reducing the number of observations identified as noise.